

1. Introduction

I began my project with a strong desire to understand the fundamentals of machine learning and progressively build more complex models. This report outlines my journey—from basic regression techniques to the development of an advanced transformer-based model for text generation. Each week presented new challenges and opportunities that deepened my understanding.

2. Week 1: Basic Regression Techniques

2.1 Linear Regression

My journey started with linear regression. I explored how to model the relationship between a dependent variable and one or more independent variables using a simple linear equation:

$$y = mx + c$$

I learned to estimate the slope (m) and the intercept (c) using the least squares method, which helped me understand how errors can be minimized using mean squared error.

2.2 Polynomial Regression

To capture non-linear relationships, I extended the linear model by incorporating polynomial features like x^2 and x^3 . This allowed me to fit curves to data that did not follow a straight-line pattern. I discovered the importance of balancing model complexity to avoid overfitting while still capturing the underlying trends in the data.

3. Week 2: Perceptron and Basic Neural Networks

3.1 My Journey with the Perceptron Model

In Week 2, I implemented a simple perceptron model from scratch using NumPy. I initialized the weights and bias to small random values and used the sigmoid function as my activation function. Training the perceptron using gradient descent provided me with a hands-on understanding of how neural networks learn from data.

3.2 Implementing Logical Gates

I applied the perceptron model to simulate basic logical operations. I trained separate models to mimic the behaviour of the AND and OR gates using their respective truth tables. I implemented a simple NOT gate using a threshold function and then combined these gates to construct the XOR gate. This process deepened my understanding of how simple neural units can be orchestrated to perform more complex logical operations.

3.3 Building a Full Adder & Ripple Carry Adder

Taking it a step further, I used the learned logical gates to build a full adder that computes the sum and carry for binary addition. I then extended this concept to create a Ripple Carry Adder, which can

add multi-bit binary numbers by cascading multiple full adders. This was an exciting milestone that showcased the potential of neural networks in simulating digital circuits.

4. Week 3: Baseline Deep Learning with Neural Networks

4.1 Transitioning to Deep Learning Frameworks

Moving beyond manually coded perceptrons, I transitioned to using deep learning frameworks like TensorFlow and Keras. This shift allowed me to leverage built-in functionalities and optimizations, significantly streamlining the model development process.

4.2 Data Preprocessing and Model Building

I focused on transforming raw data into a format suitable for training a neural network. I implemented advanced preprocessing techniques—such as normalization, shuffling, and batching using TensorFlow's `tf.data.Dataset`—to efficiently prepare input–target pairs. I then built a Multi-Layer Perceptron (MLP) model that could capture non-linear relationships without the need for manually combining simple perceptrons.

4.3 Training and Evaluation

With the MLP model in place, I trained it using automated gradient descent and backpropagation. I experimented with various hyperparameters, including the learning rate, number of hidden units, and training epochs. Evaluating the model's performance provided me with critical insights into its strengths and areas for improvement, setting a solid baseline for future, more advanced models.

5. Week 4: Advanced Transformer-Based Model for Text Generation

5.1 Understanding the Transformer Architecture

In Week 4, I dove into one of the most advanced architectures in deep learning—the transformer model. I studied its components, such as token and position embeddings, multi-head self-attention, and feed-forward networks. I learned how causal masking ensures that the model generates text in an autoregressive manner.

5.2 Assembling the Model Components

I built the transformer model using TensorFlow's Functional API. I combined token embeddings with positional embeddings and stacked multiple transformer blocks. Each block included multi-head self-attention and a feed-forward neural network, reinforced by residual connections and layer normalization. This assembly allowed the model to capture complex relationships in sequential data.

5.3 Implementing Text Generation

To showcase the capabilities of my transformer model, I implemented a text generation function. Starting with a seed sequence, the function iteratively predicted the next character, updated the sequence, and produced coherent text. This phase was particularly exciting, as it demonstrated the real-world potential of modern deep learning techniques in natural language processing.

6. Conclusion

Reflecting on my journey, I can see how far I have come—from the simplicity of linear regression and polynomial regression to the intricacies of perceptron-based logic gates, and finally, to the sophistication of transformer models for text generation. Each phase of the project has been a stepping stone, allowing me to build on previous knowledge and tackle increasingly complex challenges. This progression not only enhanced my technical skills but also deepened my appreciation for the evolution of machine learning methodologies.